

ГУАП

КАФЕДРА №

43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ст. преп.

должность, уч. степень, звание

подпись, дата

М.Д. Поляк

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Синхронизация потоков средствами POSIX

по дисциплине: Операционные системы

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4132

подпись, дата

В.С.Янгулов

инициалы, фамилия

Санкт-Петербург 2024

Задание на лабораторную работу
"Граф запуска потоков",
"Выводы"

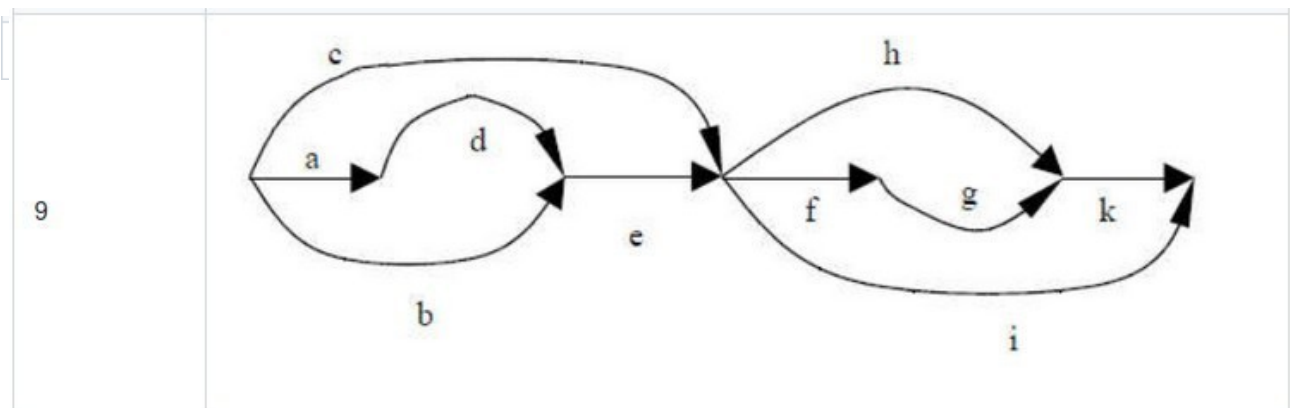
Цель работы

Знакомство с многопоточным программированием и методами синхронизации потоков средствами POSIX.

Номер варианта	Номер графа запуска потоков	Интервалы с несинхронизированными потоками	Интервалы с чередованием потоков
25	9	bcd	fhi

Индивидуальное задание

- С помощью таблицы вариантов заданий выбрать граф запуска потоков в соответствии с номером варианта. Вершины графа являются точками запуска/завершения потоков, дугами обозначены сами потоки. Длину дуги следует интерпретировать как ориентировочное время выполнения потока. В процессе своей работы каждый поток должен в цикле выполнять два действия:
 1. выводить букву имени потока в консоль;
 2. вызывать функцию `computation()` для выполнения вычислений, требующих задействования ЦП на длительное время. Эта функция уже написана и подключается из заголовочного файла `lab2.h`, изменять ее не следует.
- В соответствии с вариантом выделить на графе две группы с выполняющимися параллельно потоками. В первой группе потоки не синхронизированы, параллельное выполнение входящих в группу потоков происходит за счет планировщика задач (см. [примеры 1](#) и [2](#)). Вторая группа синхронизирована семафорами и потоки внутри группы выполняются в строго зафиксированном порядке: входящий в группу поток передает управление другому потоку после каждой итерации цикла (см. [пример 3](#) и [задачу производителя и потребителя](#)). Таким образом потоки во второй группе выполняются в строгой очередности.



Результат выполнения работы

```
obuza@Obuza:~/lab2/os-task2-Obuza$ ./a.out
abcaabcbcbdbcbcdcecececfhifhifhifhighhighikikiki
obuza@Obuza:~/lab2/os-task2-Obuza$ cd test/
```

Исходный код программы с комментариями

```
#include "lab2.h"

#include <cstring>
#include <semaphore.h>

#define THREAD_COUNT 10

pthread_t tid[THREAD_COUNT];

pthread_mutex_t lock;

sem_t semF, semH, semI, semB, semC, semG, semK;
int err;
int num_iterations = 3;


void* thread_a(void* arg);
void* thread_b(void* arg);
void* thread_c(void* arg);
void* thread_d(void* arg);
void* thread_e(void* arg);
void* thread_f(void* arg);
void* thread_h(void* arg);
void* thread_g(void* arg);
void* thread_i(void* arg);
void* thread_k(void* arg);


unsigned int lab2_thread_graph_id()
{
    return 9;
}

const char* lab2_unsynchronized_threads()
{
    return "bcd";
}

const char* lab2_sequential_threads()
{
    return "fhi";
}

void error()
{
    std::cerr << "Unable to create thread. Error: " << strerror(err) << std::endl;
}

void* thread_a(void* ptr)
```

```

{
//Интервал 1
for (int i = 0; i < num_iterations; i++)
{
//Блокируем мьютекс
pthread_mutex_lock(&lock);
std::cout << "a" <<
std::flush;
//Разблокируем мьютекс
pthread_mutex_unlock(&lock);
//Вызывается после unlock поэтому другие потоки могут выполняться параллельно во
время выполнения computation()
computation();
}
return ptr;
}

void* thread_c(void* ptr)
{
//Интервал 1
for (int i = 0; i < num_iterations; i++)
{
//Блокируем мьютекс
pthread_mutex_lock(&lock);
std::cout << "c" <<
std::flush;
//Разблокируем мьютекс
pthread_mutex_unlock(&lock);
computation();
}
//Ждём семафор B, поток будет заблокирован пока semB не станет > 0
sem_wait(&semB);
//Создаем поток D
err = pthread_create(&tid[3], NULL, thread_d, NULL);
if (err)
{
error();
return ptr;
}

//Интервал 2
for (int i = 0; i < num_iterations; i++)
{
pthread_mutex_lock(&lock);
std::cout << "c" <<
std::flush;
pthread_mutex_unlock(&lock);
computation();
}
//Ждём завершение потока B
pthread_join(tid[2], NULL);
//Создаем поток E

```

```
err = pthread_create(&tid[4], NULL, thread_e, NULL);  
if (err)
```

```

{
error();
return ptr;
}
//Интервал 3
for (int i = 0; i < num_iterations; i++)
{
pthread_mutex_lock(&lock);
std::cout << "c" <<
std::flush;
pthread_mutex_unlock(&lock);
computation();
}
//Ждём завершение потока E
pthread_join(tid[4], NULL);
//Создаем поток I
err = pthread_create(&tid[8], NULL, thread_i, NULL);
if (err)
{
error();
return ptr;
}
//Ждём завершение потока I
pthread_join(tid[8], NULL);
return ptr;
}

```

```

void* thread_d(void* ptr)
{
//Интервал 2
for (int i = 0; i < num_iterations; i++)
{
pthread_mutex_lock(&lock);
std::cout << "d" <<
std::flush;
pthread_mutex_unlock(&lock);
computation();
}
return ptr;
}

```

```

void* thread_b(void* ptr)
{
//Интервал 1
for (int i = 0; i < num_iterations; i++)
{
pthread_mutex_lock(&lock);
std::cout << "b" <<
std::flush;
pthread_mutex_unlock(&lock);
computation();
}
//Ждём завершение потока A

```

```
pthread_join(tid[0], NULL);  
//Повышаем значение семафора semB
```

```
sem_post(&semB);
```

```
//Интервал 2
```

```
for (int i = 0; i < num_iterations; i++)
```

```
{
```

```
pthread_mutex_lock(&lock);
```

```
std::cout << "b" <<
```

```
std::flush;
```

```
pthread_mutex_unlock(&lock);
```

```
computation();
```

```
}
```

```
//Ждём завершение потока D
```

```
pthread_join(tid[3], NULL);
```

```
return ptr;
```

```
}
```

```
void* thread_e(void* ptr)
```

```
{
```

```
for (int i = 0; i < num_iterations; i++)
```

```
{
```

```
pthread_mutex_lock(&lock);
```

```
std::cout << "e" <<
```

```
std::flush;
```

```
pthread_mutex_unlock(&lock);
```

```
computation();
```

```
}
```

```
return ptr;
```

```
}
```

```
void* thread_i(void* ptr)
```

```
{
```

```
//Создаем поток F
```

```
err = pthread_create(&tid[5], NULL, thread_f, NULL);
```

```
if (err)
```

```
{
```

```
error();
```

```
return ptr;
```

```
}
```

```
//Создаем поток H
```

```
err = pthread_create(&tid[7], NULL, thread_h, NULL);
```

```
if (err)
```

```
{
```

```
error();
```

```
return ptr;
```

```
}
```

```
//Интервал 4
```

```
for (int i = 0; i < num_iterations; i++)
```

```
{
```

```
//поток i ждет, пока семафор semI станет больше нуля.
```

```
sem_wait(&semI);
```

```
pthread_mutex_lock(&lock);  
std::cout << "i" <<  
std::flush;
```

```

pthread_mutex_unlock(&lock);
computation();
//увеличение значения семафора semF, сигнализация потоку f, что он может
продолжить работу.
sem_post(&semF);
}

//Создаем поток G
err = pthread_create(&tid[6], NULL, thread_g, NULL);
if (err)
{
error();
return ptr;
}
//Интервал 5
for (int i = 0; i < num_iterations; i++)
{
//поток i ждет, пока семафор semI станет больше нуля.
sem_wait(&semI);
pthread_mutex_lock(&lock);
std::cout << "i" << std::flush;
pthread_mutex_unlock(&lock);
computation();
//увеличение значения семафора semG, сигнализация потоку G, что он может
продолжить работу.
sem_post(&semG);
}
//Ждём завершение потока H
pthread_join(tid[7], NULL);
//Создаем поток K
err = pthread_create(&tid[9], NULL, thread_k, NULL);
if (err)
{
error();
return ptr;
}

//Интервал 6
for (int i = 0; i < num_iterations; i++)
{
//поток i ждет, пока семафор semI станет больше нуля.
sem_wait(&semI);
pthread_mutex_lock(&lock);
std::cout << "i" << std::flush;
pthread_mutex_unlock(&lock);
computation();
//увеличение значения семафора semK, сигнализация потоку K, что он может
продолжить работу.
sem_post(&semK);
}
//Ждём завершение потока K
pthread_join(tid[9], NULL);

```

```
//увеличение значения семафора semC, сигнализация потоку C, что он может
продолжить работу.
sem_post(&semC);
return ptr;
}
```

```
void* thread_f(void* ptr)
{
//Интервал 4
for (int i = 0; i < num_iterations; i++)
{
//поток F ждет, пока семафор semF станет больше нуля.
sem_wait(&semF);
pthread_mutex_lock(&lock);
std::cout << "f" << std::flush;
pthread_mutex_unlock(&lock);
computation();
//увеличение значения семафора semH, сигнализация потоку H, что он может
продолжить работу.
sem_post(&semH);
}
return ptr;
}
```

```
void* thread_h(void* ptr)
{
//Интервал 4
for (int i = 0; i < num_iterations; i++)
{
//поток H ждет, пока семафор semH станет больше нуля.
sem_wait(&semH);
pthread_mutex_lock(&lock);
std::cout << "h" <<
std::flush;
pthread_mutex_unlock(&lock);
computation();
//увеличение значения семафора semI, сигнализация потоку i, что он может
продолжить работу.
sem_post(&semi);
}
//Ждём завершение потока F
pthread_join(tid[5], NULL);
```

```
//Интервал 5
for (int i = 0; i < num_iterations; i++)
{
sem_wait(&semH);
pthread_mutex_lock(&lock);
std::cout << "h" <<
std::flush;
pthread_mutex_unlock(&lock);
computation();
```

```
sem_post(&seml);
```

```

}
//Ждём завершение потока G
pthread_join(tid[6], NULL);
return ptr;
}

```

```

void* thread_g(void* ptr)
{
//Интервал 5
for (int i = 0; i < num_iterations; i++)
{
sem_wait(&semG);
pthread_mutex_lock(&lock);
std::cout << "g" <<
std::flush;
pthread_mutex_unlock(&lock);
computation();
sem_post(&semH);
}
return ptr;
}

```

```

void* thread_k(void* ptr)
{
//Интервал 6
for (int i = 0; i < num_iterations; i++)
{
sem_wait(&semK);
pthread_mutex_lock(&lock);
std::cout << "k" <<
std::flush;
pthread_mutex_unlock(&lock);
computation();
sem_post(&seml);
}

return ptr;
}

```

```

int lab2_init()
{
if (pthread_mutex_init(&lock, nullptr) != 0)
{
std::cerr << "Mutex initialization failed!" << std::endl;
return 1;
}
}

```

```

if (sem_init(&semC, 0, 0) != 0 || sem_init(&semB, 0, 0) != 0 ||
sem_init(&semF, 0, 1) != 0 || sem_init(&semH, 0, 0) != 0 ||
sem_init(&seml, 0, 0) != 0 || sem_init(&semG, 0, 1) != 0 ||
sem_init(&semK, 0, 1) != 0)

```

```
{
```

```
std::cerr << "Semaphore initialization failed" << std::endl;
```

```

return 1;
}

err = pthread_create(&tid[0], nullptr, thread_a, nullptr);
if (err)
{
error();
return 1;
}
err = pthread_create(&tid[1], nullptr, thread_c, nullptr);
if (err)
{
error();
return 1;
}
err = pthread_create(&tid[2], nullptr, thread_b, nullptr);
if (err)
{
error();
return 1;
}

pthread_join(tid[1], nullptr);
pthread_mutex_destroy(&lock);
sem_destroy(&semC);
sem_destroy(&semB);
sem_destroy(&semH);
sem_destroy(&semI);
sem_destroy(&semF);
sem_destroy(&semG);
sem_destroy(&semK);

std::cout <<
std::endl; return 0;
}

```

Вывод

В ходе работы были изучены основы многопоточного программирования и методы синхронизации потоков с помощью POSIX. Были освоены создание и управление потоками, а также использование мьютексов и семафоров.